

### 1. [10pt] Boolean/CMP flags

AND 결과: AL=14h. AND는 CF=0, OF=0으로 만들고, 결과가 0이 아니므로 ZF=0, sign bit가 0이므로 SF=0, 14h는 1 bit가 2개라 PF=1.

CMP AL,20h는 14h-20h를 수행한다. 결과는 F4h이므로 CF=1, OF=0, SF=1, ZF=0. 따라서 unsigned 비교인 JB는 jump, signed 비교인 JL도 SF != OF이므로 jump.

### 2. [10pt] 조건부 점프

JA: CF=0 AND ZF=0

JB: CF=1

JBE: CF=1 OR ZF=1

JE: ZF=1

JG: ZF=0 AND SF=OF

JL: SF != OF

JGE: SF=OF

JNE: ZF=0

JO: OF=1

JS: SF=1

### 3. [10pt] 조건문

```
mov eax, VAR1
cmp eax, VAR2
jg  L_FALSE
mov eax, VAR3
cmp eax, VAR4
je  L_FALSE
L_TRUE:
    mov VAR5, 1
    jmp L_END
L_FALSE:
    mov VAR5, 0
L_END:
```

### 4. [10pt] 45배 shift 최적화

```
mov eax, var
mov ebx, eax
shl eax, 5
mov edx, ebx
shl edx, 3
add eax, edx
mov edx, ebx
shl edx, 2
add eax, edx
add eax, ebx
```

### 5. [15pt] MUL/DIV와 signed 식

(1) 옳지 않은 설명: 4. DIV의 dividend는 피연산자보다 2배 큰 AX, DX:AX, EDX:EAX 형태이며 quotient가 목적지 크기에 맞지 않으면 divide error가 발생한다.

```
mov eax, VAR1
neg eax
imul eax, 7
mov ebx, VAR2
sub ebx, VAR3
cdq
idiv ebx
mov VAR4, eax
```

### 6. [15pt] STDCALL swap

```
swap PROC
    push ebp
    mov ebp, esp
    sub esp, 4
    push eax
    push ebx
    mov eax, [ebp+8]
    mov ebx, [eax]
    mov [ebp-4], ebx
    mov ebx, [ebp+12]
    mov edx, [ebx]
    mov [eax], edx
    mov edx, [ebp-4]
    mov [ebx], edx
    pop ebx
    pop eax
    mov esp, ebp
    pop ebp
    ret 8
swap ENDP
```

### 7. [10pt] STDCALL 호출

```
push OFFSET y
push OFFSET x
call swap
```

STDCALL은 callee가 ret 8로 인자를 정리하므로 caller가 add esp,8을 수행하지 않는다.

### 8. [10pt] 실행 결과

(1) EAX=FFFFFFFFh. -1을 산술 오른쪽 shift하면 계속 FFFFFFFFh이고, 이를 왼쪽으로 1bit shift하면 FFFFFFFEh.

(2) '\*'가 5개 출력된다. 이후 AX=00FFh, BL=10h에서  $255/16$ 은 quotient=0Fh, remainder=0Fh이므로 AL=0Fh, AH=0Fh이며 divide error는 없다.

#### 9. [10pt] REAL4

(1)  $0.15625 = 1.25 * 2^{-3}$ . sign=0, exponent=124(7Ch), fraction=010... 이므로 bit pattern은 3E200000h. little endian 메모리에는 00 00 20 3E.

(2) 3FC00000h는 sign=0, exponent=127, fraction=.5이므로  $1.5 * 2^0 = 1.5$ .

#### 10. [10pt] 메모리 관리

정답: 1, 3, 4, 5

HeapAlloc으로 할당한 메모리는 stack frame 해제와 무관하므로 직접 HeapFree 등으로 반환해야 한다.